AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH
408/1, Kuratoli, Khilkhet, Dhaka 1229, Bangladesh

Project Title: Focus-Flow: Priority Task Manager	
Course Title: Programming In Python	Date of Submission: 28/03/2026
Course Code: CSC4162	Section: B
Semester: Spring 2025-2026	Course Teacher: MD. TANZEEM RAHAT

Declaration and Statement of Authorship:

1. I/we hold a copy of this Assignment/Case-Study, which can be produced if the original is lost/damaged.
2. This Assignment/Case-Study is my/our original work and no part of it has been copied from any other student's work or from any other source except where due acknowledgement is made.
3. No part of this Assignment/Case-Study has been written for me/us by any other person except where such collaborationhas been authorized by the concerned teacher and is clearly acknowledged in the assignment.
4. I/we have not previously submitted or currently submitting this work for any other course/unit.
5. This work may be reproduced, communicated, compared and archived for the purpose of detecting plagiarism.
6. I/we give permission for a copy of my/our marked work to be retained by the Faculty for review and comparison, including review by external examiners.
7. I/we understand thatPlagiarism is the presentation of the work, idea or creation of another person as though it is your own. It is a formofcheatingandisaveryseriousacademicoffencethatmayleadtoexpulsionfromtheUniversity. Plagiarized material can be drawn from, and presented in, written, graphic and visual form, including electronic data, and oral presentations. Plagiarism occurs when the origin of them arterial used is not appropriately cited.
8. I/we also understand that enabling plagiarism is the act of assisting or allowing another person to plagiarize or to copy my/our work.

* *Student(s) must complete all details except the faculty use part.*
** Please submit all assignments to your course teacher or the office of the concerned teacher.

Group Name/No.:

No	Name	ID	Program	Signature
1	**MD JAHIDUN MUNTAKA FARABI**	22-49201-3	CSE	Farabi

Table Of Contents

1.0 INTRODUCTION 3
2.0 OOP IMPLEMENTATION 3
2.1 Task Class (Entity Class) 3
2.2 TaskManager Class (Service Class) 3
2.3 Storage Class (Data Access Class) 4
2.4 Main Class (Controller Class) 4
3.0 TECHNICAL SPECIFICATIONS 4
3.1 Programming Language 4
3.2 Environment & Package Management 4
3.3 External Libraries 4
3.4 Data Storage (Persistence) 4
4.0 KEY FEATURES 5
4.1 Automated Priority Scoring 5
4.2 Dynamic Sorting & Visualization 5
4.3 Case-Insensitive Search 5
4.4 Data Persistence (JSON) 5
4.5 Robust Input Validation 5
4.6 Productivity Summary Report 5
5.0 System Screenshots 5
6.0 CONCLUSION 8
7.0 GITHUB REPOSITORY LINK 8

1.0 INTRODUCTION

In today's fast-paced environment, managing numerous tasks while maintaining focus on high-priority goals is a significant challenge. To address this, Focus-Flow: Priority Task Manager was developed as a specialized Command Line Interface (CLI) tool. The primary objective of this project is to provide a structured way for users to record, organize, and prioritize their daily tasks using a mathematical logic based on the Eisenhower Matrix.

The system moves beyond simple "To-Do" lists by introducing a Priority Scoring mechanism. Every task is evaluated based on two key metrics: Urgency and Importance. By calculating the product of these two factors, the application automatically ranks tasks, ensuring that users always see their most critical work at the top of the list.

Key goals achieved through this project include:

- Persistent Data Management: Ensuring that tasks are stored safely in JSON format and are never lost between sessions.
- Object-Oriented Design: Implementing a modular architecture using Python classes to ensure scalability and clean code.
- User-Centric Sorting: Providing an automated sorting system to remove the mental burden of deciding what to do next.
- Robust Interaction: Handling user inputs gracefully with validation to ensure a crash-free experience.

By integrating these features, Focus-Flow serves as a reliable productivity assistant that empowers users to manage their time more effectively through data-driven prioritization.

2.0 OOP IMPLEMENTATION

The Focus-Flow project is developed using the core principles of Object-Oriented Programming (OOP). The system is divided into four distinct classes, each with a specific responsibility. This modular approach makes the code clean, reusable, and easy to debug.

2.1 Task Class (Entity Class)

The Task class is the basic data model of the application. It encapsulates all the information related to a single task.

- Attributes: task_id, title, urgency, importance, and priority_score.
- Logic: It automatically calculates the priority_score by multiplying Urgency and Importance.

2.2 TaskManager Class (Service Class)

The TaskManager class contains the "Business Logic" of the application. It manages the collection of Task objects.

- Key Functions:
    - Add/Delete: Handles adding new tasks and removing them by ID.

    - Sorting: Implements the logic to sort tasks based on their priority score in descending order.
    - Search: Allows users to find specific tasks using keywords.

2.3 Storage Class (Data Access Class)

This class is responsible for Data Persistence. It ensures that the user's data is not lost when the program is closed.

- Functionality: It handles reading from and writing to the tasks.json file. It converts Python objects into JSON format and vice-versa.

2.4 Main Class (Controller Class)

The Main class acts as the interface between the user and the system logic.

- Functionality: It runs the main loop, displays the menu using the rich library, takes user input, and performs Input Validation (using try-except blocks) to ensure the program doesn't crash on invalid data.

3.0 TECHNICAL SPECIFICATIONS

To meet the professional standards of Python development, the project utilizes several key technologies and tools. These ensure that the application is stable, organized, and easy to install on any machine.

3.1 Programming Language

- **Python 3.12:** The project is written entirely in Python, taking advantage of its simple syntax and powerful Object-Oriented features.

3.2 Environment & Package Management

- **Virtual Environment (.venv):** A dedicated virtual environment was created to isolate the project's dependencies from the global system. This ensures that the libraries used in this project do not conflict with other Python projects.
- **Pip & Requirements.txt:** All external dependencies are managed via pip. A requirements.txt file is included so that anyone can install the necessary tools with a single command (pip install -r requirements.txt).

3.3 External Libraries

- **Rich Library:** This library is used to create a professional Command Line Interface. It allows the application to display data in colorful, organized tables and provides stylized console output.

3.4 Data Storage (Persistence)

- **JSON (JavaScript Object Notation):** The project uses a tasks.json file to store data. JSON was chosen because it is lightweight, easy to read, and integrates perfectly with Python dictionaries to save and load task information.

4.0 KEY FEATURES

Focus-Flow includes several advanced features that make it more than a simple list-making tool. Each feature is designed to improve the user's productivity and ensure data integrity.

4.1 Automated Priority Scoring

The core logic of the application is its scoring system. Instead of manual sorting, it calculates a **Priority Score** for every task.

Priority Score = Urgency * Importance

This ensures that tasks that are both urgent and important stay at the top of the user's view.

4.2 Dynamic Sorting & Visualization

The application automatically sorts the task list in **descending order** (highest score to lowest). Using the rich library, this data is displayed in a colorful, professional table that makes it easy to distinguish between different tasks at a glance.

4.3 Case-Insensitive Search

Users can search for specific tasks by title. The search is designed to be **case-insensitive**, meaning searching for "study" will find "Study", "STUDY", or "sTuDy".

4.4 Data Persistence (JSON)

To ensure a professional experience, the app features **Automatic Saving**. All changes (adding or deleting tasks) are written to a tasks.json file. When the app restarts, it loads the previous data exactly where the user left off.

4.5 Robust Input Validation

The application is protected against user errors. By using try-except blocks, the app handles invalid inputs (such as entering a letter when a number is required) without crashing, providing clear error messages instead.

4.6 Productivity Summary Report

The app provides a statistical breakdown, including the **Total Number of Tasks**, the number of **High-Priority Tasks** , and the **Average Priority Score** of the current list.

5.0 System Screenshots

Figure 1: Focus-Flow Main Interface and Menu.

🎯 Focus-Flow: Priority Task Manager
1. Add New Task
2. View Sorted Tasks
3. Search Tasks
4. Delete Task
5. View Summary Report
6. Exit & Save

Select an option (1-6): 2

Priority Task List

ID	Task Name	Score
1	Python lab task	25
5	Python Quiz-1	25
2	Adv web tech quiz	20
6	Python Lab 1 task	12
4	ABC	8

Figure 2: Task List sorted by Priority Score (Highest to Lowest).

🎯 Focus-Flow: Priority Task Manager
1. Add New Task
2. View Sorted Tasks
3. Search Tasks
4. Delete Task
5. View Summary Report
6. Exit & Save

Select an option (1-6): 5

📊 Productivity Summary
Total Tasks: 5
High Priority Tasks: 3
Avg Priority Score: 18.0

Figure 3: Real-time Productivity Summary Report.

```
🎯 Focus-Flow: Priority Task Manager
1. Add New Task
2. View Sorted Tasks
3. Search Tasks
4. Delete Task
5. View Summary Report
6. Exit & Save

Select an option (1-6): 1
Enter Task Title: ABCD
Enter Urgency (1-5): xyz
Invalid input! Please enter numbers only.
```

Figure 4: Input Validation handling invalid data types.

.

```
🎯 Focus-Flow: Priority Task Manager
1. Add New Task
2. View Sorted Tasks
3. Search Tasks
4. Delete Task
5. View Summary Report
6. Exit & Save

Select an option (1-6): 4
Enter Task ID to delete: 4
Task deleted.
```

Figure 5: Task Deletion by ID

FocusFlow-Python-Project Public

Pin Watch 0

main 1 Branch 0 Tags Go to file t Add file Code

jahidunfarabi Final UI integration and project completion de80e4b · 4 hours ago 5 Commits

.gitignore	Project setup and Task entity class	5 days ago
main.py	Final UI integration and project completion	4 hours ago
manager.py	Integrated TaskManager with sorting and reporting logic	4 hours ago
requirements.txt	Project setup and Task entity class	5 days ago
storage.py	Implementation of Storage class for JSON persistence	2 days ago
task.py	Fixed Task class init and updated data storage	4 hours ago
tasks.json	Fixed Task class init and updated data storage	4 hours ago

Figure 6: Version Control History on GitHub.

6.0 CONCLUSION

The **Focus-Flow: Priority Task Manager** project successfully demonstrates the application of Object-Oriented Programming (OOP) in solving real-world productivity challenges. By organizing the code into modular classes like Task, TaskManager, and Storage, the system achieves high readability and maintainability.

Through this development process, several key programming concepts were practiced:

- **Encapsulation:** Keeping data and logic together within classes.
- **Data Persistence:** Managing real-time file I/O using JSON.
- **Error Resilience:** Implementing robust validation to enhance user experience.
- **Version Control:** Maintaining a structured 5-day workflow using Git and GitHub.

In conclusion, Focus-Flow is not just a simple task list; it is a reliable tool that empowers users to make data-driven decisions about their daily priorities, ensuring they focus on what truly matters.

7.0 GITHUB REPOSITORY LINK

The complete source code, including the version history and environment setup, is available on GitHub. You can access the repository using the link below:

GitHub Link: https://github.com/jahidunfarabi/FocusFlow-Python-Project.git